

"""

## **# POINTS TO NOTE**

- 1) ORDERED
- 2) CHANGEABLE OR MUTABLE
- 3) ALLOW DUPLICATES
- 4) ACCEPTS ALL DATA TYPES
- 5) LIST WITHIN LIST
- 6) LIST FRAMING
- 7) CHANGE LIST ITEMS
- 8) BUILTIN FUNCTION (APPEND, INSERT....)

"""

## **# LIST EXAMPLES**

```
mylistno=[1,20,30]
```

```
mylistname=["hai",'hello','ever','never']
```

```
mylistfloat=[10.25,100.35,25.69]
```

```
mylistcomplex=[2+5j,4+9j]
```

```
print(mylistno)
```

```
print(mylistname)
```

```
print(mylistfloat)
print(mylistcomplex)
```

## **#ACCEPT MIXED DATA TYPE**

```
mylistmix=["welcome",100,10.25,2+5j, True]
print(mylistmix)
print(type(mylistmix))
```

## **#OTHER WAY TO CONSTRUCT LIST**

```
mylistway=list(("strings",10.25,200,2+5j))
print(mylistway)
```

## **#LIST WITHIN LIST**

```
mylistwithin=[[1,3,5],[2,4,6],[3,6,9]]
print(mylistwithin[1]) # ACCESS LIST ITEMS
```

## **# LIST FRAMING**

```
"""
```

0 1 2 3 4 5 6... POSITIVE INDEXING

..... -7 -6 -5 -4 -3 -2 -1 NEGATIVE INDEXING

"""

## **# ACCESSING THE ITEMS FROM THE LIST USING SLICING OPERATOR(:)**

```
print(mylistno[1])
```

```
print(mylistname[3])
```

```
print(mylistway[2])
```

```
print(mylistno[1:3]) # items at index 1&2 will be printed not 3
```

```
print(mylistwithin[:2])
```

```
print(mylistfloat[1:])
```

```
print(mylistwithin[-2:-1])
```

## **#CHECKING THE ITEM IN LIST**

```
if "welcome" in mylistmix:
```

```
    print("yes, 'welcome' is in the mixed list")
```

```
else:
```

```
    print("no such found")
```

## **#CHANGE LIST ITEMS**

```
thislist=["tulips","tomato","tapioca"]
```

```
thislist[1]="hibiscus"
```

```
print(thislist)
```

## **#CHANGE RANGE OF ITEM VALUES**

```
flowers=["rose","potato","beet"]
```

```
flowers[1:3]=["jasmine","marrygold"] #limit change 1,2...
```

```
print(flowers)
```

```
flowers[1:3]=["jasmine"] # index 2 left blank
```

```
print(flowers)
```

## **#BUILTIN FUNCTIONS IN LIST**

**#ADD OR APPEND : ADD ITEMS TO LIST** : append() and also add list to list: don't return value

'''

Syntax of List append()

The syntax of the append() method is:

**list.append(item)** '''

```
fruits=["apple","orange","mango"]
```

```
animals=["dog","cat","elephant"]
```

''' items to list '''

```
fruits.append("grapes")
```

```
print(fruits) → ["apple", "orange", "mango", "grapes"]
```

''' list to list '''

```
fruits.append(animals)
```

```
print(fruits) → ["apple", "orange", "mango", ["dog", "cat", "elephant"]]
```

**#EXTEND THE LIST:** xyz.extend(abc) : xyz and abc are two diff list :  
don't return value

'''

Syntax of List extend()

The syntax of the extend() method is:

```
list1.extend(iterable) '''
```

```
fruits=["apple", "orange", "mango"]
```

```
season=["guava", "pears"]
```

```
fruits.extend(season)
```

```
print(fruits)
```

```
" EXTEND LIST,TUPLE,SETS"
nations=["india","usa","uae"]
programs_tup=("c","python","java")
lang_sets={"tamil","english","telugu"}
nations.extend(programs_tup)
print("list_tup: ",nations)
nations.extend(lang_sets)
print("list_tup_sets: ",nations)
```

#### # DIFFERENCE BETWEEN APPEND AND EXTEND

```
a=[1,2,3]
b=[4,5,6]
c=(2,8,6)
a.append(b)
print(a)
a.append(c)
print(a)
d=['a','d','r']
e=['6','9','7']
f=('*','/','+')
d.extend(e)
```

```
print(d)
d.extend(f)
print(d)
```

## #OTHER WAYS TO EXTEND THE LIST

```
a=[1,2]
b=[3,4]
a+=b # a=a+b
print('a =',a)
```

```
c=[4,5]
print(len(c))
d= [8,5]
c[len(c):] =b
print('c =',c)
```

**# INSERT THE ITEMS:** xyz.insert(index,item) : ITEMS ARE INSERTED AND NOT DELETED

# don't return value

'''

Syntax of List insert()

The syntax of the insert() method is

```
list.insert(i, elem)
```

insert() Parameters

The insert() method takes two parameters:

index - the index where the element needs to be inserted

element - this is the element to be inserted in the list

```
'''
```

```
fruits=["apple","orange","mango"]
```

```
fruits.insert(1,"guava")
```

```
print(fruits)
```

```
subjects_tup=('eng','mat','phy')
```

```
marks_set={50,78,96}
```

```
fruits.insert(4,subjects_tup)
```

```
print(fruits)
```

```
fruits.insert(5,marks_set)
```

```
print(fruits)
```



# POP : TO REMOVE ITEM FROM LIST: XYZ.POP(INDEX)

# no value returned

'''

Syntax of List pop()

The syntax of the pop() method is:

list.pop(index)

'''

```
fruits=["apple","orange","mango"]
```

```
fruits.pop(0)
```

```
print(fruits)
```

```
poped_one=fruits.pop(0)
```

```
print('pope_done:',poped_one)
```

```
print('updated list: ',fruits)
```

```
lang=['c','python','fotron','pascal','r']
```

```
print('when no index passed:',lang.pop())
```

```
lang=['c','python','fotron','pascal','r']  
print('when positiv.index given : ',lang.pop(2))
```

```
lang=['c','python','fotron','pascal','r']  
print('when negative index given: ',lang.pop(-1))
```

```
lang=['c','python','fotron','pascal','r']  
print('neg.index.pass: ',lang.pop(-3))
```

**# INDEX : index()**

# return value if nothing is found in the given condition

```
animals=['dog','cat','camel']
```

```
a=animals.index('cat')
```

```
print(a)
```

```
'''
```

list index() parameters

The list index() method can take a maximum of three arguments:

element - the element to be searched

start (optional) - start searching from this index

end (optional) - search the element up to this index

Return Value from List index()

The index() method returns the index of the given element in the list.

If the element is not found, a ValueError exception is raised.

**Note: The index() method only returns the first occurrence of the matching element.**

```
'''
```

```
vowels=['a','e','i','o','u']
```

```
index=vowels.index('e')
print('the index of e : ',index)
ind=vowels.index('p') # return error p is not found
print(ind)"""
```

**# WORKING OF INDEX WITH START AND STOP PARAMETER**

```
alpha=['a','c','e','y','n','u','t','c','q']
index=alpha.index('c',3) # c after 3rd index in searched if repeated or
not
print(index)
ews=alpha.index('y',0,4) # y between index 2 and 4 is searched
print(ews)
```

**# REMOVE: xyz.remove()**

'''

Syntax of List remove()

The syntax of the remove() method is:

list.remove(element)

remove() Parameters

The remove() method takes a single element as an argument and removes it from the list.

If the element doesn't exist, it throws ValueError: list.remove(x): x not in list exception.

do not return value

'''

```
animals=['cat','dog','rabbit']
```

```
animals.remove('rabbit')
```

```
print('updated list: ',animals)→ ['cat', 'dog']
```

```
animals=['cat','dog','rabbit','dog','cow','dog']
```

```
animals.remove('dog')
```

```
print('list without dog',animals) # in output only first occurred dog is removed
```

''' error formed becoz goat not in list

```
animals=['cat','dog','rabbit']
```

```
animals.remove('goat')
```

```
print(animals)
```

'''

**# COUNT: RETURN THE NO OF TIMES ITEM PRESENT IN LIST**

''''''

Syntax of List count()

The syntax of the count() method is:

```
list.count(element)
```

count() Parameters

**The count() method takes a single argument:**

element - the element to be counted

Return value from count()

The count() method returns the number of times element appears in the list.

```
"""
```

```
numbers = [2, 3, 5, 2, 11, 2, 7]
```

```
count = numbers.count(2)
```

```
print('Count of 2:', count)
```

```
vowels=['a','e','i','o','a','e','u']
```

```
count=vowels.count('a')
```

```
print(count)
```

```
ind=vowels.count('p') # no error gen...op is 0
```

```
print(ind)
```

```
""" count tuple and list inside list """
```

```
random=['a',('a','b'),('c','z'),[1,8]]
```

```
count1=random.count(('a','b'))
```

```
print(count1)
```

```
count2=random.count([1,8])  
print(count2)
```

## **# REVERSE**

'''

Syntax of List reverse()

The syntax of the reverse() method is:

```
list.reverse()
```

reverse() parameter

The reverse() method doesn't take any arguments.

Return Value from reverse()

The reverse() method doesn't return any value. It updates the existing list.

'''



```
os=['windows','linux','mac_os']
```

```
os.reverse()
```

```
print('reversed one :',os)
```

```
""" reverse using slicing operator """
```

```
os=['windows','linux','mac_os']
```

```
rev_list=os[::-1] #[start:stop:step]
```

```
print(rev_list)
```

```
""" using for loop """
```

```
os=['windows','linux','mac_os']
```

```
for o in reversed(os):
```

```
    print(o)
```

## **# SORT arrange in ascending or decending order**

'''

### **sort() Syntax**

The syntax of the sort() method is:

```
list.sort(key=..., reverse=...)
```

Alternatively, you can also use Python's built-in sorted() function for the same purpose.

```
sorted(list, key=..., reverse=...)
```

Note: The simplest difference between sort() and sorted() is: sort() changes the list directly and

doesn't return any value, while sorted() doesn't change the list and returns the sorted list.

'''

```
prime=[11,3,5,7,23]
```

```
prime.sort()
```

```
print(prime) # ascending order
```

'''

### **sort() Parameters**

By default, `sort()` doesn't require any extra parameters. However, it has two optional parameters:

`reverse` - If `True`, the sorted list is reversed (or sorted in Descending order)

`key` - function that serves as a key for the sort comparison

`sort()` Return Value

The `sort()` method doesn't return any value. Rather, it changes the original list.

If you want a function to return the sorted list rather than change the original list, use `sorted()`.

```
'''
```

```
''' decending order '''
```

```
vowels=['e','o','u','a','i']
```

```
vowels.sort(reverse=True)
```

```
print(vowels)
```

# copy :

'''

copy() Syntax

The syntax of the copy() method is:

```
new_list = list.copy()
```

copy() Parameters

The copy() method doesn't take any parameters.

copy() Return Value

The copy() method returns a new list. It doesn't modify the original list.

'''

```
mylist=['cat',23,6.78]
```

```
new_list=mylist.copy()
```

```
print(new_list)
```

```
mylist=['cat',23,6.78]
```

```
mynew=mylist
```

```
print(mynew)
```

```
""" appending in new list """
```

```
old_list=[1,2,9]
```

```
new_form=old_list
```

```
new_form.append('dog')
```

```
print(new_form)
```

```
# CLEAR
```

```
"""
```

Syntax of List clear()

The syntax of clear() method is:

```
list.clear()
```

The clear() method removes all items from the list.

clear() Parameters

The clear() method doesn't take any parameters.

Return Value from clear()

The clear() method only empties the given list. It doesn't return any value.

```
'''
```

```
prime=[3,5,7,23,11]
```

```
prime.clear()
```

```
print(prime)
```

```
''' under clear(),list of combo of list,tup,set everything clears out completely'''
```

```
# join list in python
```

```
list1 = ["a", "b", "c"]
```

```
list2 = [1, 2, 3]
```

```
list3 = list1 + list2
```

```
print(list3)
```

```
list1 = ["a", "b", "c"]
```

```
list2 = [1, 2, 3]
```

```
for x in list2:  
    list1.append(x)  
  
print(list1)
```

```
list1 = ["a", "b", "c"]  
list2 = [1, 2, 3]
```

```
list1.extend(list2)  
print(list1)
```

## List Methods

Python has a set of built-in methods that you can use on lists.

| Method                                | Description                                                                  |
|---------------------------------------|------------------------------------------------------------------------------|
| <a href="#"><code>append()</code></a> | Adds an element at the end of the list                                       |
| <a href="#"><code>clear()</code></a>  | Removes all the elements from the list                                       |
| <a href="#"><code>copy()</code></a>   | Returns a copy of the list                                                   |
| <a href="#"><code>count()</code></a>  | Returns the number of elements with the specified value                      |
| <a href="#"><code>extend()</code></a> | Add the elements of a list (or any iterable), to the end of the current list |
| <a href="#"><code>index()</code></a>  | Returns the index of the first element with the specified value              |
| <a href="#"><code>insert()</code></a> | Adds an element at the specified position                                    |
| <a href="#"><code>pop()</code></a>    | Removes the element at the specified position                                |
| <a href="#"><code>remove()</code></a> | Removes the item with the specified value                                    |



[reverse\(\)](#) Reverses the order of the list

[sort\(\)](#) Sorts the list